

The Living Map: Spatial Memory for Emergency Robots

TSYP14 Technical Challenge — IEEE RAS × IEEE AESS, Tunisia Section Chapters

Team Winek?

Phase 1 — October 5, 2026

1 Problem and approach

When a robot explores a mine, its knowledge disappears if it fails or loses power, and no GPS or network reaches inside. We target the *Mines / Tunnels* environment (mostly 1-D corridors with branches and a single entrance) and detect two event types: **hazardous gas** and **victim detected** (a third, *blocked path*, is reserved in the packet format).

Key idea: the memory lives in the beacons, not in the robot. A **Writer** robot explores autonomously and drops small RF beacons at every event, every junction and every 10 m, forming a chain linked by next-hop pointers. When it finishes or runs low on battery it returns to the entrance and uploads its log to the Outside Network Area (ONA). If the Writer dies, the beacons still hold the map. A later **Executor** robot is briefed at the entrance and follows the beacons to continue the mission instead of starting from zero.

2 System architecture

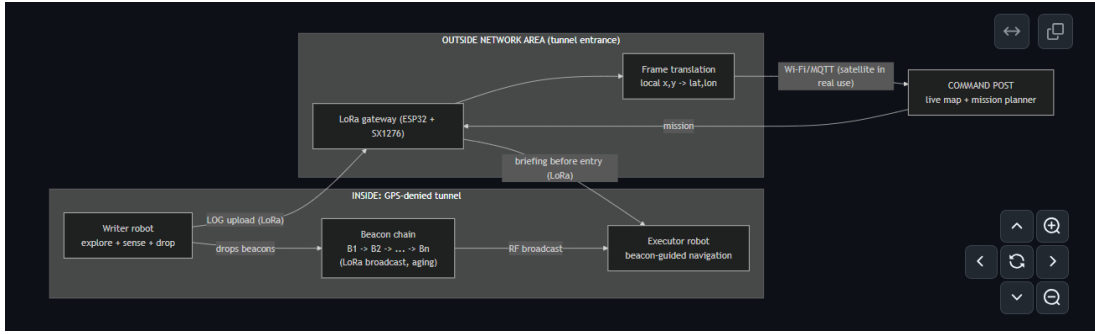


Figure 1: Architecture. Everything between the mine and the outside passes through the ONA gateway.

Hard rule respected: **the robots never talk to the command post.** Writer and Executor use LoRa (SX1276 on ESP32, 868 MHz band with 433 MHz as fallback; the legal band in Tunisia is being confirmed with the regulator) to the ONA gateway only; the gateway (ESP32 + LoRa) forwards to the command post over Wi-Fi/MQTT, with satellite as the real-deployment option. Beacons are dumb one-way broadcasters: they do not relay, which keeps them cheap and long-lived.

3 Data flow

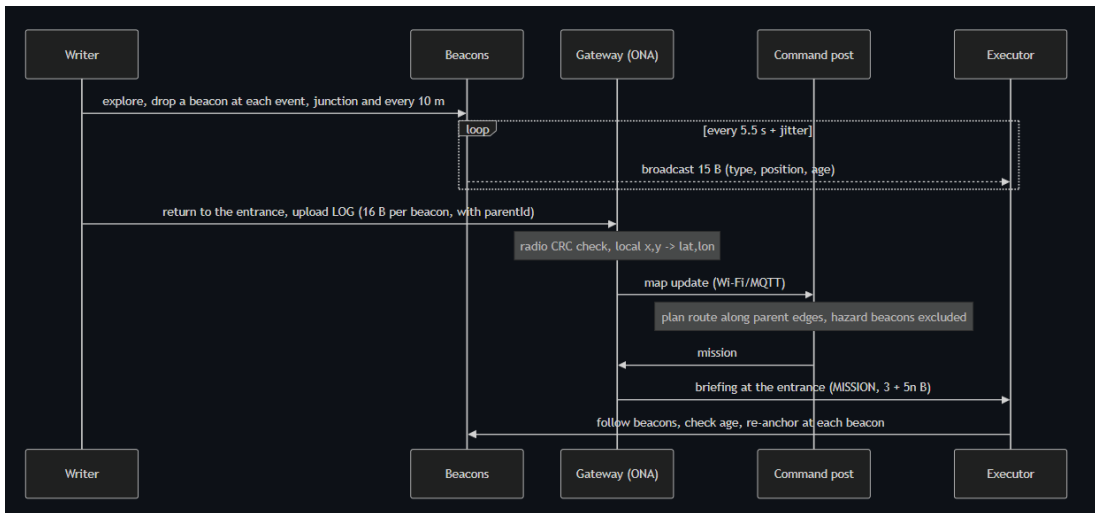


Figure 2: End-to-end data flow, implemented in the simulation message bus.

4 Writer autonomy, event detection and beacon deposition

The Writer explores by frontier-based depth-first search over the corridor graph: at each junction it takes an unvisited branch, backtracks at dead ends, and keeps a private frame (origin at the entrance, $+x$ in its starting direction). It has no GPS: position comes from dead reckoning (wheel odometry plus IMU), so it drifts; the simulation models a heading bias with random walk and a 1% scale error.

What to record, and when. A beacon is dropped (a) at the entrance, (b) at every junction or dead end, (c) at every victim or gas detection, and (d) every 10 m of corridor so gaps stay short. On gas detection (sensor range 5 m) the Writer drops a hazard beacon before the plume and turns back; it never enters the hazard zone. The drop mechanism on the physical robot is a servo-based dropper (Phase 2).

5 Beacon message and signal design

The beacon frame is **15 bytes little-endian**: a 1-byte frame type prefix followed by the 14-byte `BeaconMsg` payload. The format is implemented identically in the Python simulation (`livingmap/beacon.py`) and the ESP32 firmware (`firmware/include/beacon_msg.h`); a test compares the Python output with a frame produced by the compiled firmware header.

Off	Sz	Field	Meaning
0	1	type	Frame type: 1 = BEACON (2 = LOG, 3 = MISSION)
1	1	version	Format version, currently 1
2	1	beaconId	This beacon (1..254)
3	1	eventType	0 waypoint, 1 victim, 2 gas, 3 blocked (reserved)
4	1	nextHopId	Next beacon in drop order (0xFF = none)
5	2	x_cm	int16, local x in cm (range ± 327 m)
7	2	y_cm	int16, local y in cm
9	2	bearingDeg	int16, 0..359, Writer heading at drop time
11	4	ageAtWriteS	uint32, seconds since the event was recorded

Table 1: Beacon frame payload (`BeaconMsg`, 14 B) with the 1 B type prefix.

Frame	Layout	Size	Direction
LOG	[type=2] [BeaconMsg:14] [parentId:1]	16 B	Writer $\rightarrow$ gateway, at the entrance
MISSION	[type=3] [targetEvent:1] [n:1] [entries:5n]	$3 + 5n$ B, $n \leq 48$	Gateway $\rightarrow$ Executor, at the entrance

Table 2: Other frames. A MISSION entry is `{beaconId:u8, x_cm:i16, y_cm:i16}` (5 B each). `parentId` is the beacon it was reached from, so the command post can rebuild the tunnel tree after the Writer backtracks.

Design choices. (i) 15 B at SF7/125 kHz gives 46 ms airtime, so beacons can broadcast every few seconds without hogging the channel. (ii) Centimetre integers instead of floats: two bytes per axis, and encode/decode is identical on the ESP32 and in Python. (iii) Local coordinates only: beacons never hold GPS, so no beacon needs a GPS fix and the robots stay in their private frame; the gateway translates. (iv) An **age counter** instead of a timestamp: the beacon has no clock and no synchronisation step is needed, it counts seconds since it was written (kept across deep sleep). (v) **No application CRC**: the SX1276 appends and checks a hardware CRC (`radio.setCRC(true)` on every node); frames that fail it are dropped, saving 2 bytes. (vi) **Next-hop pointers plus coordinates**: beacons are numbered in drop order and each points to the next one; the route itself comes from the briefing (the command post follows the LOG `parentId` edges), because the next beacon in drop order can be on another branch after a dead end.

Aging mechanism

A beacon cannot be updated after it is dropped, so trust decays with age. The reader computes

$$\text{confidence} = \exp\left(-\frac{\text{ageAtWriteS}}{\tau}\right)$$

with $\tau = 600$ s for gas, 1800 s for victim and blocked, and no decay for waypoints. Below **0.3** the information is *stale*: the Executor still uses the beacon as a landmark but treats the hazard as “verify before trusting” and re-checks it with its own sensors. In the simulation the Executor reaches the beacons about 20 min after they were written: the gas beacon is at 0.13 (stale) and the victim beacon at 0.51 (fresh).

Radio link parameters

Identical on every node (Writer, Executor, beacons, gateway):

- Frequency: 868 MHz band (433 MHz fallback); the legal band in Tunisia is still to be confirmed.
- SF7 / BW 125 kHz / CR 4/5, sync word 0x12, preamble 8 symbols, TX power 14 dBm.
- Airtime: 15 B = 46.3 ms, 16 B = 51.5 ms, a 43 B mission = 87 ms. A 1 % duty cycle allows one beacon frame every 4.6 s per node.
- Rebroadcast: every 5.5 s plus a random 0.2 s, about 0.7 % duty cycle.
- No acknowledgements: beacons are simple one-way broadcasters. Initial settings to be tuned by range tests in Phase 2.

6 Drop mechanism and power (prototype)

Drop mechanism. A vertical tube holds the beacons in ID order and a servo-driven trapdoor releases one at a time. The Writer first sends a WRITE packet addressed to the next beacon ID (three times, at 0 dBm), then opens the trapdoor for 400 ms. It then expects to hear the beacon's first broadcast within about 10 s; otherwise it retries once and logs a failure. Beacons are pre-numbered in drop order, so each one can point to the next. The prototype briefing is a 4-byte message (first beacon ID, target event).

Power. A programmed beacon is awake 0.15 s per cycle at about 125 mA and asleep at about 0.05 mA, giving an average of 3.8 mA, i.e. about 263 h (11 days) on 1000 mAh. These currents are datasheet-based assumptions to be measured on the bench. The prototype uses 3 beacons, while the simulation drops 13 to exercise a longer mine.

7 Simulation results

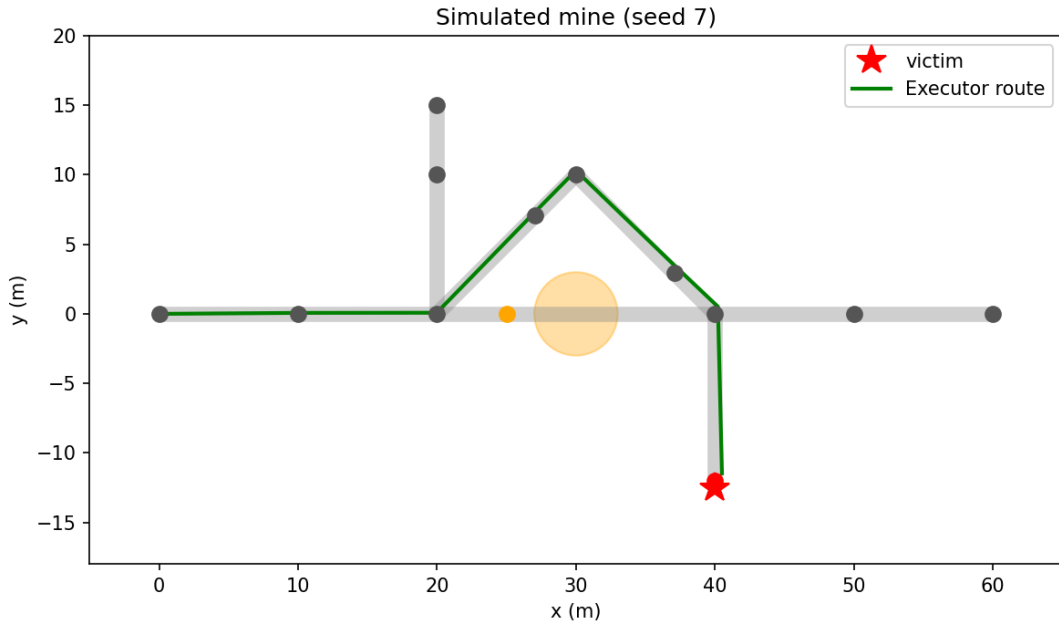


Figure 3: Simulated mine: loop with a gas plume on the direct corridor, a dead end, and a victim branch.

Metric (seed 7)	Result
Beacons dropped	13 (11 waypoints, 1 gas, 1 victim)
Writer drift on return to entrance	1.38 m after about 250 m of travel
Route chosen by command post	beacons 1–2–3–7–8–9–10–11; gas beacon 6 excluded, bypass used instead of the direct corridor
Executor delay / result	deployed 15 min later, reached the victim beacon
Executor gas exposure	0 steps; CRC errors injected at 10 % were rejected and retried

Table 3: Simulation results at seed 7.

Run it with `py run_demo.py`; the automated tests (`py -m pytest`, 31 tests) check mission success, zero gas exposure and that no direct link to the command post exists. The simulation uses straight corridors on a graph and a simplified sensor model; obstacle avoidance and RF propagation are left for the physical prototype.

8 Innovation

1. **Memory in the environment.** Knowledge survives the robot: beacons are numbered in drop order and each points to the next, so the Executor can follow the chain inward from the first beacon even if the Writer never returns.
2. **Self-describing, aging beacons.** Each 15-byte frame carries type, position, heading and an age counter, so stale hazard data is flagged instead of blindly trusted.
3. **Drift-free relay of position.** The Executor re-anchors on every beacon, so the Writer's dead-reckoning errors do not accumulate over a mission.
4. **Frame separation.** Robots never use GPS; only the gateway translates, so a wrong GPS anchor degrades only the displayed map.
5. **Low-cost, dumb beacons** that only broadcast, keeping the hardware cheap and the system robust.

9 Failure cases and implementation plan (summary)

Failure cases (full table in `docs/failure_cases.md`): Writer dies or is stuck (beacons still hold the map); beacon lost or dead (bounded search, then dead reckoning to the next known beacon); RF collisions and CRC errors (jitter, retry, hardware CRC drop); stale data (aging and re-sensing); dead-reckoning drift (re-anchoring); bad GPS anchor or heading (only the map is affected); beacon reboot or age counter reset (RTC memory, brown-out detection); gateway or uplink down (buffering, backup uplink); hazard blocking the only route (command post reports no safe route).

Implementation plan (full plan in `docs/implementation_plan.md`): two roles, a software lead (packet, frames, command post, gateway software, documentation) and an embedded lead (firmware, LoRa, sensors, chassis, beacon dropper). Eight weeks to 01/12/2026: parts and LoRa link (weeks 1–2), Writer base, command post and event sensing (3–4), Executor navigation (5), full integration (6), failure tests (7), documentation and pitch (8), with the riskiest items scheduled first.